qPCRtools: a cross-platform C++ desktop application for qPCR data analysis with validated equivalence to its R implementation

Xiang Li[1,2,*] Huichuan Huang[1,2,*]

[1] State Key Laboratory for Conservation and Utilization of Bio-Resources in Yunnan, Yunnan Agricultural University, Kunming, China

[2] Key Laboratory for Agro-Biodiversity and Pest Control of Ministry of Education, Yunnan Agricultural University, Kunming, China

[*] Corresponding authors: lixiang117423@gmail.com (X.L.), absklhhc@gmail.com (H.H.)

Present address (X.L.): Yazhouwan National Laboratory, Sanya, Hainan, China

Abstract

Quantitative real-time PCR (qPCR) is a standard technique for measuring gene expression, yet turning instrument-exported C_q values into publication-ready results requires a chain of non-trivial steps: amplification-efficiency estimation, choice of quantification method, statistical comparison of groups, and figure preparation. The established tools for this chain are predominantly programming-based—including our own R package *qPCRtools*—and therefore remain out of reach for part of the laboratory community, notably students and wet-lab researchers without programming experience; instrument-vendor packages and free stand-alone or web-based tools cover individual steps, but tie the analysis to one instrument platform, to one step, or to uploading the data to a remote server. Here we present *qPCRtools*, a cross-platform desktop application written in C++17 with Qt6 that exposes the analytical core of the R package through a point-and-click interface and ships as a single executable requiring no R, Python, or other runtime environment. The application implements standard-curve construction and amplification-efficiency estimation, the ΔC_t, $\Delta\Delta C_t$ and efficiency-based (RqPCR) relative quantification methods, group comparison by two-sample t-test, Wilcoxon rank-sum test, or

one-way ANOVA with Tukey's HSD letter grouping, and one-click export of the summary table, the per-replicate raw data and a plain-text analysis record of the inputs and parameters used. We validated the C++ implementation against the R package on identical inputs: across every shared quantification method the compared quantities agree to floating-point precision, and the validation is reproducible from the open scripts shipped with the software. Preparing the port also revealed and corrected several defects in result aggregation, each covered by a regression test. *qPCRtools* is freely available under the MIT license for Windows and macOS, and its source code compiles on Linux.

1 Introduction

Quantitative real-time PCR (qPCR) is among the most widely used techniques in molecular biology for quantifying nucleic acids and measuring gene expression [5]. A sound qPCR analysis involves several interdependent steps: estimating primer amplification efficiency, choosing an appropriate quantification strategy, performing statistical testing, and producing publication-quality figures [4]. When the amplification efficiencies of target and reference genes are comparable, the $2^{-\Delta\Delta C_T}$ method [8] is convenient; when they differ, an efficiency-corrected or standard-curve-based approach is required instead [11]. These analytical choices, combined with the need for rigorous statistics and clear visualization, make qPCR data analysis demanding for many laboratory researchers.

A range of tools address qPCR analysis [10], yet each covers a different part of the workflow (Table 1). Scripting-based implementations—our own *qPCRtools* [7] and *pcr* [1]—implement the calculation chain transparently, but require programming skill and a configured software environment. Software distributed with real-time instruments (for example QuantStudio Design & Analysis and CFX Maestro) offers a graphical workflow but is tied to that instrument platform. Free stand-alone programs cover individual steps: REST [12] computes efficiency-based relative expression for two-group comparisons with a randomisation test, LinRegPCR [13, 15] estimates baseline and amplification efficiency from raw fluorescence, and qBase [6] implements relative quantification together with the geNorm stability measure [17] for selecting reference genes. Free web applications integrate several of these steps—SATQPCR [14], DAG Expression [3] and Auto-qPCR [9]—but they require uploading the data to a remote server (or deploying the web stack locally), which constrains offline use and the handling of unpublished data. The gap we address is therefore not a missing algorithm but a miss-

ing delivery mode: a tool that combines *validated numerical equivalence* to a published implementation with a *native, cross-platform, zero-dependency* graphical workflow.

Table 1: Delivery mode and functionality of representative qPCR analysis tools. • provided, ○ not provided, ○/• partial or version-dependent (statistics, for example, were added in the later qBase+ release, and vendor software differs between instruments and versions). Feature sets were compiled from the cited papers, the vendors' documentation and the survey of Pabinger *et al.* [10]. [a] *qPCRtools* [7], *pcr* [1]. [b] e.g. QuantStudio Design & Analysis, CFX Maestro. [c] SATQPCR [14], DAG Expression [3], AutoqPCR [9]. REST [12], LinRegPCR [13, 15], qBase [6], geNorm [17].

Tool	Free	No coding	Offline	Efficiency	Statistics	Ref. genes
R packages[a]	•	○	•	•	•	•
This application	•	•	•	•	•	•
REST	•	•	•	•	•	○
LinRegPCR	•	•	•	•	○	○
qBase	•	•	•	•	○/•	•
Instrument software[b]	•	•	•	•	○/•	○
Web apps[c]	•	•	○	•	•	•

Statement of need

The programming requirement is not a marginal obstacle. In our own teaching and collaborative work, the users who most need a defensible qPCR analysis are frequently the ones least able to install an R environment, resolve package dependencies, and write the few lines of code that the analysis requires: newly enrolled students, visiting researchers, and wet-lab staff whose projects run for a single season. They currently fall back either on manual spreadsheet calculations—which are difficult to audit and easy to get wrong—or on whichever commercial instrument software happens to be installed. Our R package [7] already provides the algorithms, and its adoption since 2022 (more than 30 citations) shows that the analytical choices it implements are accepted in published work, but that adoption is bounded by the R environment itself.

We therefore reimplemented the analytical core as a native desktop application. The design goal is deliberately narrow: keep the validated behaviour of the R implementation, and remove every prerequisite except a double-click.

We introduce *qPCRtools*, a C++/Qt6 desktop application that reimplements the analytical core of our R package behind a point-and-click interface distributed as a single executable. This Application Note makes three contributions: (i) a C++/Qt architecture in which a tested core algorithm library is decoupled from a web-based user interface; (ii) a validation demonstrating numerical equivalence to the R package across all shared quantification methods, shipped as a reproducible validation package that a reader can re-run; and (iii) statistical and functional extensions beyond the original R version, including non-parametric tests, effect sizes, outlier removal, and automatic reference-gene selection. Preparing the port also revealed defects in result aggregation that are shared with the R package—the input of the reference-gene stability measure, replicate handling in the efficiency method, and the $\Delta\Delta C_t$ calibrator—as well as one specific to the C++ implementation (multiple-comparison p-values); all were corrected and are covered by regression tests.

2 Implementation

2.1 Architecture

qPCRtools is written in C++17 and built on Qt6 [16], which provides cross-platform abstractions and the native application shell. The user interface is rendered with web technologies—Bootstrap 5.3 for layout, with Apache ECharts [2] bundled for the figure view—hosted inside a `QWebEngineView`, and exchanges data with the native backend through Qt WebChannel via a registered bridge object. This separation lets the presentation layer iterate rapidly as HTML, CSS, and JavaScript while all numerical computation stays in compiled native code. The analytical core is isolated as a standalone static library, `qpcr_core`, which depends only on Qt Core and Eigen and is independent of the GUI; it is exercised directly by the unit-test suite and is also linked by the command-line equivalence driver described in Results. Statistical distribution functions use the GNU Scientific Library (GSL) for accuracy. The resulting interface is shown in Figure 1.

2.2 Quantification algorithms

The core reimplements the analytical methods of the R package. From a dilution series, amplification efficiency is obtained by ordinary least squares of C_q on $\log_d$(concentration), i.e. on the number of d-fold dilution steps (d = dilution factor, 4 in the bundled example), yielding slope s, intercept, R^2,

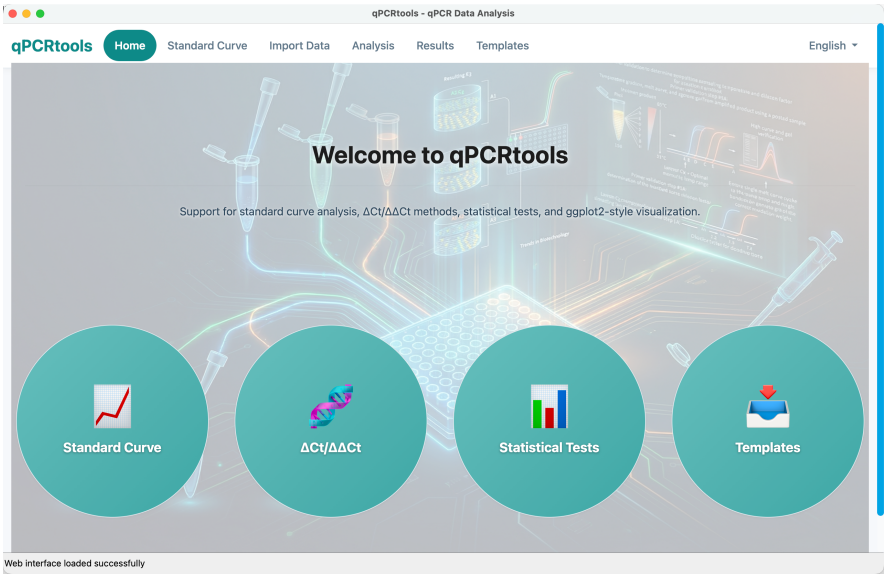


Figure 1: The *qPCRtools* desktop interface (version 1.1.0; screen captured from the running application). A single cross-platform window hosts the web-based UI: the home screen gives access to the four analysis areas (standard curve, $\Delta C_t/\Delta\Delta C_t$, statistical tests, and templates for data entry). The interface is bilingual (Chinese/English); the English language is shown here.

and a regression p-value; efficiency is

$$E = d^{-1/s} - 1, \tag{1}$$

with s expressed per $\log_d$ unit (equivalently, per dilution step). Relative expression is computed by the ΔC_t and $\Delta\Delta C_t$ methods [8],

$$\mathrm{RE} = 2^{-\left[(C_{q,\mathrm{target}} - C_{q,\mathrm{ref}})_{\mathrm{treated}} - (C_{q,\mathrm{target}} - C_{q,\mathrm{ref}})_{\mathrm{control}}\right]}, \tag{2}$$

and by an efficiency-based (RqPCR) method [14] in which per-sample quantities are $E^{C_{q,\min} - C_q}$ and are normalized by the geometric mean of one or more reference genes [11].

2.3 Statistical analysis

For each target gene the application compares groups statistically. Group comparisons are performed on the linear relative-expression scale—the scale on which the values are reported and plotted, and the one used by the R

implementation—rather than on ΔC_t; the choice is stated here explicitly because the two scales give different p-values for the same data, and a single scale is therefore applied throughout the application and the validation. The core library implements two-sample t-tests with and without the Welch variance correction, the paired t-test, the Wilcoxon rank-sum and signed-rank tests, and one-way ANOVA followed by Tukey's HSD post-hoc letter grouping, and it reports Cohen's d effect sizes alongside the t-test results; it also supports IQR-based outlier removal and automatic reference-gene selection by the geNorm algorithm [17]. These options extend the original R package, which provided t-test and ANOVA/Tukey testing only. The graphical interface currently exposes the Welch two-sample t-test, the Wilcoxon rank-sum test and one-way ANOVA with Tukey's HSD; the paired t-test and the Wilcoxon signed-rank test are exercised by the test suite but are not yet offered as interface options. For multi-group designs, Tukey's HSD controls the family-wise error rate across the pairwise comparisons of a gene; the reported p-values are per gene and are not additionally corrected across the many genes analysed in one run, which is stated as a limitation rather than left implicit.

2.4 Software engineering and scope

All results are presented as tables that can be exported as CSV or Excel for further plotting, and groups that differ significantly are labelled with Tukey letters directly in the result table. Version 1.1.0 does not render figures: the ECharts-based figure view is present in the code base but disabled, and the figures of this paper were therefore produced from the application's numerical output by the plotting script shipped in the repository (`validation/plot_examples.py`), not by the graphical interface. Each analysis can also be exported as a plain-text *analysis record* that lists the input files and their row counts, the reference gene(s), the control group, the statistical test, the outlier-removal setting, the software version and a timestamp, so that a reader can re-run the same analysis without inspecting the session. *qPCRtools* is managed with CMake and ships with a unit-test suite; the interface is bilingual (Chinese/English) and reads CSV input as well as Excel (`.xlsx`) workbooks when built against the optional OpenXLSX dependency. The application is distributed as a single self-contained executable for Windows and macOS, requiring no R, Python, or runtime installation, and the source compiles on Linux. It is released under the MIT license. Two R functions are not yet ported—the standard-curve slope–intercept quantification (`CalExpCurve`) and the reverse-transcription reagent volume

calculation (`CalRTable`)—and are planned for a future release.

3 Results

3.1 Typical analysis workflow

To illustrate a typical analysis, we analyzed the bundled example dataset (`examples/cq.csv` and `examples/design.csv`), comprising the target gene *fos-glo-myc* normalized against the reference gene *Beta Actin* across three treatment groups (0, 0.5, 1) with three biological replicates each. Using the $\Delta\Delta C_t$ method with a Welch two-sample t-test against the control group (0), relative expression rose monotonically to 1.92 at dose 0.5 and 2.74 at dose 1, both significantly different from the control ($p = 0.006$ and $p = 0.022$; Figure 2). As defined by Livak and Schmittgen, the calibrator is the control group, so the control group is ≈ 1 (here 1.01 ± 0.21). The group means, standard deviations and p-values quoted here are reproduced without any dependency on the application or on the R package by `validation/check_example_stats.R`. Applied to a four-fold dilution series of four genes, the standard-curve module recovered slopes between -2.07 and -2.24 C_q per four-fold dilution step ($\log_4$ scale) with $R^2 > 0.99$ for three of the four primers (efficiencies 0.85–0.95), illustrating amplification-efficiency estimation that, per MIQE guidance, determines which quantification method is appropriate. The complete workflow—import, configuration, analysis, and export—is performed through the graphical interface without writing code.

3.2 Equivalence to the R package

We validated the C++ implementation against the original R package [7] by running both on identical inputs for every shared method—$\Delta\Delta C_t$ (`CalExp2ddCt`), the standard curve (`CalCurve`), and the efficiency-based RqPCR method (`CalExpRqPCR`)—and comparing every resulting quantity pairwise (Table 2, Figure 3). Inputs were drawn from the repository's example data and from a dilution series used in the original R package; both implementations read the same on-disk files, so any difference reflects the implementations alone.

For the two expression methods, $\Delta\Delta C_t$ and RqPCR, every compared quantity—expression values, standard deviations, and standard errors—matched between implementations to within floating-point precision (maximum relative difference $< 4 \times 10^{-15}$). For the standard curve, slope, in-

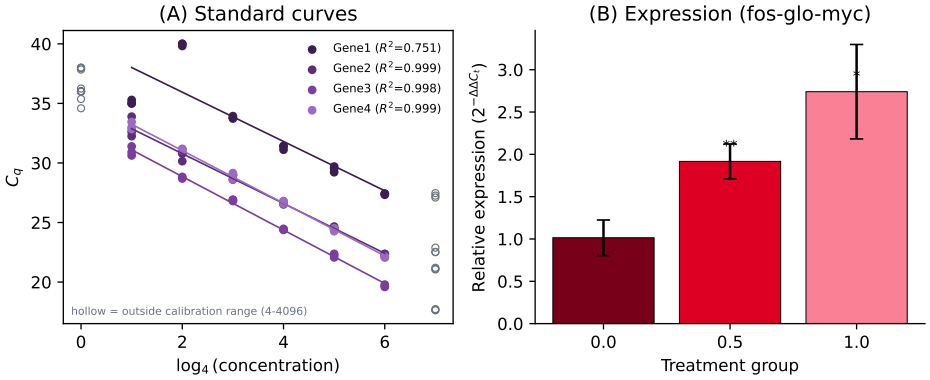


Figure 2: Analysis output on real data. (A) Standard curves for four genes over a four-fold dilution series, with fitted regressions and R^2; the abscissa is $\log_4$ concentration (dilution steps), matching the regression, and hollow symbols mark wells outside the calibrated range (4–4096) that are excluded from the fit. (B) Relative expression of *fos-glo-myc* across treatment groups ($\Delta\Delta C_t$ method, calibrated to the control group), with significance labels. Both panels were drawn from the numerical output of the application (the standard-curve parameters and the per-group means exported by the results table) by the plotting script shipped in the repository; version 1.1.0 exports result tables rather than rendering figures.

tercept, R^2, and efficiency agreed to within 2.5×10^{-3} relative difference; the residual is attributable to the R package rounding its tabulated output to two to four decimal places, since the underlying C_q inputs are identical. Plotting every C++ result against its R counterpart, all points fall on the line $y = x$ (Figure 3). These results confirm that the C++ reimplementation preserves the analytical behavior of the R package across all shared methods. The validation is fully reproducible from the open scripts in `validation/` (see Availability).

4 Availability

Version 1.1.0 of *qPCRtools* is described here. The source code and pre-built binaries for Windows and macOS are available at `https://github.com/lixiang117423/qPCRtools_cpp` under the MIT license; no registration is required to download or run the software. The repository includes documentation, a unit-test suite, example datasets, and a fully reproducible

Table 2: Equivalence of C++ and R results across all shared methods on identical inputs. Relative difference is $|R - \text{C++}|/|R|$.

Method	Compared quantities	Pairs	Max. rel. diff.
$\Delta\Delta C_t$	expression, SD	6	3.8×10^{-15}
Standard curve	slope, intercept, R^2, efficiency	16	2.5×10^{-3}
RqPCR	expression, SE	16	2.1×10^{-15}

equivalence-validation package (`validation/`) comprising the R script, a C++ command-line driver linked against `qpcr_core`, and a comparison script that regenerates all validation outputs and figures. The project builds with CMake (≥ 3.20), requires a C++17 compiler and Qt 6 (≥ 6.5) with the WebEngine and WebChannel modules, and optionally links Eigen3, GSL, and OpenXLSX.

Authors' contributions

XL and HH designed this study and performed the analysis. XL wrote the manuscript. Both authors read and approved the final manuscript.

Acknowledgements

The authors thank the developers of the R packages and libraries on which this work builds, and the users who provided feedback during development.

Funding

This work was supported by the National Key R&D Program of China (2021YFD1000200), the National Natural Science Foundation of China (31801792, 31960554), and the Fok Ying Tung Education Foundation.

Conflict of interest

The authors declare no competing interests.

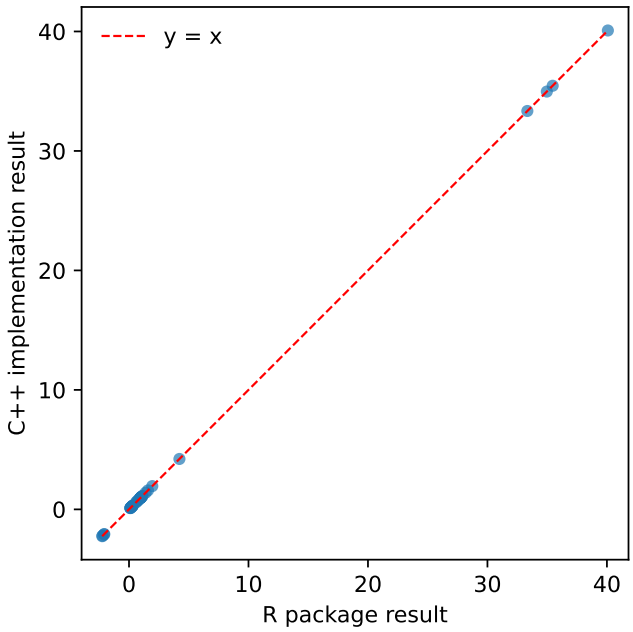


Figure 3: Numerical equivalence of the C++ implementation and the R package. Each C++ result is plotted against the corresponding R result across all shared methods ($\Delta\Delta C_t$, standard curve, RqPCR); all points fall on the $y = x$ line.

References

[1] Mohaned Ahmed and Dae-Ryong Kim. pcr: an R package for quality assessment, analysis and testing of qPCR data. *PeerJ*, 6:e4473, 2018.

[2] Apache Software Foundation. Apache ECharts: A declarative framework for rapid construction of web-based visualization. `https://echarts.apache.org/`, 2024.

[3] Maria Ballester, Raquel Cordón, and Josep M. Folch. DAG Expression: high-throughput gene expression analysis of real-time PCR data using standard curves for relative quantification. *PLoS ONE*, 8(11):e80385, 2013.

[4] Stephen A. Bustin, Jean-François Beaulieu, Jim Huggett, Ramnath Jaggi, Frederick S. B. Kibenge, Pal A. Olsvik, Lois C. Pennington,

and Tobias Toegl. MIQE précis: practical implementation of minimum standard guidelines for fluorescence-based quantitative real-time PCR experiments. *BMC Molecular Biology*, 11:74, 2010.

[5] Stephen A. Bustin, Vladimir Benes, Jeremy A. Garson, Jan Hellemans, Jim Huggett, Mikael Kubista, Reinhold Mueller, Tania Nolan, Michael W. Pfaffl, Gregory L. Shipley, et al. The MIQE guidelines: minimum information for publication of quantitative real-time PCR experiments. *Clinical Chemistry*, 55(4):611–622, 2009.

[6] Jan Hellemans, Geert Mortier, Anne De Paepe, Frank Speleman, and Jo Vandesompele. qBase relative quantification framework and software for management and automated analysis of real-time quantitative PCR data. *Genome Biology*, 8(2):R19, 2007.

[7] Xiang Li, Yingmin Wang, Jingyu Li, Xinyue Mei, Yixiang Liu, and Huichuan Huang. qPCRtools: an R package for qPCR data processing and visualization. *Frontiers in Genetics*, 13:1002704, 2022.

[8] Kenneth J. Livak and Thomas D. Schmittgen. Analysis of relative gene expression data using real-time quantitative PCR and the $2^{-\Delta\Delta C_T}$ method. *Methods*, 25(4):402–408, 2001.

[9] Gilles Maussion, Rhalena A. Thomas, Iveta Demirova, Gracia Gu, Eddie Cai, Carol X.-Q. Chen, Narges Abdian, Theodore J. P. Strauss, Sabah Kelaï, Angela Nauleau-Javaudin, Lenore K. Beitel, Nicolas Ramoz, Philip Gorwood, and Thomas M. Durcan. Auto-qPCR; a Python-based web app for automated and reproducible analysis of qPCR data. *Scientific Reports*, 11:21293, 2021.

[10] Stephan Pabinger, Stefan Rödiger, Anja Kriegner, Klemens Vierlinger, and Andreas Weinhäusel. A survey of tools for the analysis of quantitative PCR (qPCR) data. *Biomolecular Detection and Quantification*, 1:23–33, 2014.

[11] Michael W. Pfaffl. A new mathematical model for relative quantification in real-time RT-PCR. *Nucleic Acids Research*, 29(9):e45, 2001.

[12] Michael W. Pfaffl, Graham W. Horgan, and Leo Dempfle. Relative expression software tool (REST) for group-wise comparison and statistical analysis of relative expression results in real-time PCR. *Nucleic Acids Research*, 30(9):e36, 2002.

[13] Christian Ramakers, Jan M. Ruijter, Ronald H. Lekanne Deprez, and Antoon F. M. Moorman. Assumption-free analysis of quantitative real-time polymerase chain reaction (PCR) data. *Neuroscience Letters*, 339(1):62–66, 2003.

[14] Corinne Rancurel, Thuy Van Tran, Corinne Elie, and Frédérique Hilliou. SATQPCR: website for statistical analysis of real-time quantitative PCR data. *Molecular and Cellular Probes*, 46:101418, 2019.

[15] Jan M. Ruijter, Christian Ramakers, Willem M. H. Hoogaars, Yann Karlen, Olaf Bakker, Maurice J. B. van den Hoff, and Antoon F. M. Moorman. Amplification efficiency: linking baseline and bias in the analysis of quantitative PCR data. *Nucleic Acids Research*, 37(6):e45, 2009.

[16] The Qt Company. Qt Cross-Platform Application Development Framework. `https://www.qt.io/`, 2024. Version 6.x.

[17] Jo Vandesompele, Katleen De Preter, Filip Pattyn, Bruce Poppe, Nicole Van Roy, Anne De Paepe, and Frank Speleman. Accurate normalization of real-time quantitative RT-PCR data by geometric averaging of multiple internal control genes. *Genome Biology*, 3(7):research0034, 2002.